Azure Synapse + Azure Data Factory Student Lab Guide

Detailed portal steps, configuration values, scripts, and GitHub Actions CI/CD for a budget-safe Azure for Students lab.

Scope note: this guide covers the major practical features of Azure Synapse, Azure Data Factory, and Git-based CI/CD that you can realistically practice on Azure for Students. It does not cover every Azure service or every Synapse/ADF feature.

1. Before you start

Use Azure for Students, not Azure for Students Starter. Azure for Students gives you USD 100 credit for 12 months and access to the full catalog up to your credit. If you exhaust the credit, the subscription is disabled until you upgrade or renew when eligible. Keep this lab small and delete the temporary test environment after CI/CD validation.

Recommended learning outcome:

- ADLS Gen2 raw / curated / serving zones
- ADF linked services, datasets, parameters, expressions, Lookup, ForEach, If, Copy, Mapping Data Flow, trigger, Monitor
- Synapse workspace, serverless SQL, OPENROWSET, views, external data source, external file format, CETAS
- Git integration in both ADF and Synapse
- GitHub Actions deployments for ADF and Synapse artifacts

Cost guardrails

- Use Synapse serverless SQL as the primary query engine. It is pay-per-query and has no always-on cluster cost.
- Run Mapping Data Flow only a few times. It uses scaled-out Spark compute and is the most likely part of this lab to burn credit quickly.
- Keep the synthetic dataset small, stay in one region, and prefer Parquet in curated and serving zones.
- Create the test ADF and test Synapse workspace only when you are ready to validate CI/CD, then delete them.

2. Parameter sheet - fill these first

Item	Recommended value	Notes
Subscription	Azure for Students	Use the student subscription with available credit
Region	Use one region where Storage, Data Factory, and Synapse are all available	Keep everything in the same region
Resource group	rg-student-retail-lab	Use one RG for the dev environment
Storage account	stretaillab<unique>	Must be globally unique; lowercase letters and numbers
Filesystem/container	retaildatalake	ADLS Gen2 filesystem
Synapse workspace	syn-retail-dev	Dev workspace
ADF factory	adf-retail-dev	Dev ADF
Key Vault	kv-retail-lab	For secret practice and linked service integration
Test Synapse workspace	syn-retail-test	Create only for CI/CD validation
Test ADF factory	adf-retail-test	Create only for CI/CD validation

GitHub repo	azure-student-retail-lab	Holds scripts and GitHub Actions
Filesystem folders	raw/, config/, curated/, serving/	Create exactly as listed later

3. Target architecture

Flow: local synthetic files -> ADLS raw zone -> ADF metadata-driven ingestion -> ADLS curated Parquet -> ADF Mapping Data Flow -> Synapse serverless SQL analytics views -> CETAS serving outputs -> GitHub Actions deployment to test.

```
retaildatalake/
  raw/
    customers/
    products/
    orders/
    order_items/
    payments/
    returns/
    web_events/
  config/
  curated/
    customers/
    products/
    orders/
    order_items/
    payments/
    returns/
    web_events/
  serving/
    sales_daily/
    customer_360/
    product_performance/
```

4. Portal setup - detailed steps and exact values

Step 4.1 - Create the resource group

Portal field	Set this value	Why
Subscription	Azure for Students	Use your student credit
Resource group name	rg-student-retail-lab	Consistent naming
Region	Same region for all core resources	Reduces latency and complexity
Tags	Optional: project=retail-lab, env=dev	Useful for organization

Portal path: Azure Portal -> Resource groups -> Create.

Step 4.2 - Create the ADLS Gen2 storage account

Portal path: Azure Portal -> Storage accounts -> Create.

Tab	Field	Recommended value
Basics	Subscription	Azure for Students
Basics	Resource group	rg-student-retail-lab
Basics	Storage account name	stretaillab<unique>
Basics	Region	Same as resource group
Basics	Primary service	Azure Blob Storage or StorageV2 / General-purpose v2
Basics	Performance	Standard
Basics	Redundancy	Locally-redundant storage (LRS)
Advanced	Hierarchical namespace	Enable
Advanced	Access tier	Hot
Networking	Public network access	Enabled from all networks for this lab
Security	Secure transfer required	Enabled
Security	Minimum TLS version	1.2
Data protection	Blob soft delete	Optional 7 days
Data protection	Versioning	Off to keep cost and clutter low
Encryption	Microsoft-managed keys	Default is fine for the lab

After the account is created, open it and create one filesystem/container named retaildatalake under Data storage -> Containers.

Important: hierarchical namespace must be enabled to use ADLS Gen2 features.

Step 4.3 - Create the Key Vault

Portal path: Azure Portal -> Key vaults -> Create.

Field	Recommended value	Notes
Name	kv-retail-lab	Must be globally unique
Region	Same as other resources	Keep one region
Pricing tier	Standard	Enough for this lab
Permission model	Azure role-based access control	Simpler governance model
Soft delete	Leave enabled	Default protection
Purge protection	Leave default	Fine for lab

Later you will store one secret here only for practice, for example sql-admin-password or a dummy secret.

Step 4.4 - Create the Synapse workspace

Portal path: Azure Portal -> Synapse workspaces -> Create.

Tab	Field	Recommended value
Basics	Subscription	Azure for Students
Basics	Resource group	rg-student-retail-lab
Basics	Managed resource group	Leave auto-created
Basics	Workspace name	syn-retail-dev
Basics	Region	Same as storage
Basics	Data Lake Storage Gen2 account	stretaillab<unique>
Basics	File system name	retaildatalake
Security	SQL admin login	synsqladmin (example)
Security	SQL admin password	Strong password, store it in Key Vault too
Networking	Managed virtual network	Leave default / disabled for easiest

		student lab
Networking	Allow connections from all IP addresses	Enable for lab simplicity
Review + create	Validation	Create the workspace

If the workspace creator is not also an Owner on the storage account, Azure may fail to automatically grant the Synapse workspace managed identity the Storage Blob Data Contributor role. If that happens, assign it manually from the storage account IAM page.

Step 4.5 - Create Azure Data Factory

Portal path: Azure Portal -> Data factories -> Create.

Field	Recommended value	Notes
Subscription	Azure for Students	Use same subscription
Resource group	rg-student-retail-lab	Use same RG
Name	adf-retail-dev	Dev factory
Region	Same region as storage and Synapse	Keep one region
Version	V2	Current ADF generation

After creation, open Launch Studio.

Step 4.6 - Create temporary test resources for CI/CD later

Do not create the test ADF and test Synapse until you are ready to validate deployment. This avoids unnecessary cost. When ready, create the same resources again with names adf-retail-test and syn-retail-test in the same subscription, preferably the same region.

5. Mandatory access assignments

Use IAM in the Azure portal to assign these roles:

Target resource	Principal	Role
Storage account	Synapse workspace managed identity	Storage Blob Data Contributor
Storage account	ADF managed identity	Storage Blob Data Contributor
Key Vault	ADF managed identity	Key Vault Secrets User (if using Azure RBAC)
Storage account	Your own user account	Storage Blob Data Contributor or Owner while building the lab

If you use GitHub Actions with a service principal, that service principal also needs deployment permissions on the target resource group.

6. GitHub repository setup

Create a GitHub repository named azure-student-retail-lab and add these folders:

```
azure-student-retail-lab/
  data-gen/
  synapse/sql/
  cicd/
  docs/
  .github/workflows/
```

The package that accompanies this PDF already contains the starter files in this layout.

7. Dataset generation

File included: data-gen/generate_retail_data.py

Run locally:

```
python generate_retail_data.py
```

Expected output folder:

```
retail_demo/
  customers.csv
  products.csv
  orders.csv
  order_items.csv
  payments.csv
  returns.csv
  web_events.json
  ingestion_control.csv
```

8. Upload files to ADLS

Use the storage account portal or Azure Storage Explorer. Create the folders exactly as below inside the retaildatalake filesystem and upload each file into its matching folder.

Local file	Upload destination
customers.csv	raw/customers/customers.csv
products.csv	raw/products/products.csv
orders.csv	raw/orders/orders.csv
order_items.csv	raw/order_items/order_items.csv
payments.csv	raw/payments/payments.csv
returns.csv	raw/returns/returns.csv
web_events.json	raw/web_events/web_events.json
ingestion_control.csv	config/ingestion_control.csv

9. Azure Data Factory - exact artifact build steps

Open ADF Studio.

Step 9.1 - Enable Git

Field	Recommended value
Repository type	GitHub
Repository	azure-student-retail-lab
Collaboration branch	main
Root folder	/
Publish branch	adf_publish
Import existing resources	Enable if you configure Git after creating resources

Step 9.2 - Create linked service ls_adls_retail

Path: Manage -> Linked services -> New -> Azure Data Lake Storage Gen2.

Property	Value
Name	ls_adls_retail
Authentication method	System-assigned managed identity
Storage account selection	stretaillab<unique>
Test connection	Should succeed after role assignment

Step 9.3 - Create linked service ls_keyvault

Path: Manage -> Linked services -> New -> Azure Key Vault.

Property	Value
Name	ls_keyvault
Base URL	Select your Key Vault
Authentication	Managed identity
Test connection	Should succeed after Key Vault role assignment

Step 9.4 - Create datasets

Create the following datasets under Author -> Datasets. Use the same linked service ls_adls_retail for all storage-backed datasets.

Dataset name	Connector/format	Key settings
ds_control_csv	ADLS Gen2 + DelimitedText	File path = config/ingestion_control.csv; First row as header = true; Column delimiter = comma
ds_source_csv_param	ADLS Gen2 + DelimitedText	Parameters: folderPath, fileName; First row as header = true
ds_source_json_param	ADLS Gen2 + JSON	Parameters: folderPath, fileName; Use line-delimited JSON / set of objects as appropriate in the UI
ds_sink_parquet_param	ADLS Gen2 + Parquet	Parameter: folderPath

For parameterized datasets, point the path fields to dynamic content instead of hard-coded folders. Example: folderPath uses @dataset().folderPath and fileName uses @dataset().fileName.

Step 9.5 - Build pipeline pl_metadata_ingest

Create a new pipeline named pl_metadata_ingest.

- Activity 1: Lookup_Control
- Activity 2: ForEach_Entity
  - Inside ForEach: If_CSV
    - Inside true branch: Copy_CSV_To_Parquet
    - Inside false branch: Copy_JSON_To_Parquet

Lookup_Control settings

Property	Value
Name	Lookup_Control
Dataset	ds_control_csv
First row only	False

ForEach_Entity settings

Property	Value
Name	ForEach_Entity

Items	@activity('Lookup_Control').output.value
Batch count	1 initially; increase later only if needed

If_CSV settings

Property	Value
Name	If_CSV
Expression	@equals(item().source_format,'csv')

Copy_CSV_To_Parquet settings

Section	Set this
Source dataset	ds_source_csv_param
Source folderPath	@item().source_folder
Source fileName	@concat(item().source_entity,'.csv')
Sink dataset	ds_sink_parquet_param
Sink folderPath	@item().sink_folder
Sink format	Parquet
Mapping	Use default mapping first; add explicit mapping later only if needed

Copy_JSON_To_Parquet settings

Section	Set this
Source dataset	ds_source_json_param
Source folderPath	@item().source_folder
Source fileName	@concat(item().source_entity,'.json')
Sink dataset	ds_sink_parquet_param
Sink folderPath	@item().sink_folder
Sink format	Parquet

Debug this pipeline until all curated folders are populated with Parquet.

Step 9.6 - Build Mapping Data Flow df_orders_analytics

Create a new Mapping Data Flow named df_orders_analytics.

Node	Configuration
Source 1	curated/orders Parquet
Source 2	curated/order_items Parquet
Source 3	curated/payments Parquet
Join 1	orders join order_items on order_id
Join 2	joined result join payments on order_id
Derived Column	line_amount = toDecimal(quantity * unit_price * (1 - discount_pct), 18, 2); sales_date = toDate(order_date)
Filter	order_status != 'Cancelled'
Aggregate 1	Group by sales_date, sales_channel; orders = countDistinct(order_id); customers = countDistinct(customer_id); total_revenue = sum(line_amount)
Aggregate 2	Group by customer_id; total_orders = countDistinct(order_id); total_revenue = sum(line_amount)
Sink 1	serving/sales_daily/ as Parquet
Sink 2	serving/customer_360/ as Parquet

Use a small integration runtime/core count in Data Flow debug if the portal gives you sizing options. Run this only when the curated layer already exists.

Step 9.7 - Build pipeline pl_transform_publish

Property	Value

Pipeline name	pl_transform_publish
Activity	Data Flow activity
Data flow	df_orders_analytics

Step 9.8 - Trigger and monitor

Create one schedule trigger named trg_daily_retail_ingest. Start with manual runs while building. After validation, schedule pl_metadata_ingest first and pl_transform_publish second. Monitor everything in the Monitor hub.

10. Synapse - exact build steps

Open Synapse Studio and enable Git to the same GitHub repo. Use main as the collaboration branch and workspace_publish as the publish branch.

Step 10.1 - Connect to serverless SQL

From Synapse Studio, open Develop -> SQL scripts. Select the built-in serverless SQL endpoint. Every Synapse workspace includes this endpoint by default.

Step 10.2 - Run the included SQL scripts in this order

Order	Script file	What it does
1	00_create_db.sql	Creates retail_serverless database and schemas
2	01_validate_openrowset.sql	Validates direct reads from curated Parquet
3	02_views.sql	Creates reusable external views over curated files
4	03_analytics_views.sql	Creates business views such as sales daily and customer 360
5	04_external_objects.sql	Creates master key, database scoped credential, external data source, and external file format
6	05_cetas_publish.sql	Writes serving query outputs back to ADLS with CETAS

Before running scripts 01-05, replace every <storage-account> placeholder with your actual storage account name.

Step 10.3 - SQL validation checks

```
SELECT TOP 10 * FROM ext.vw_orders;

SELECT TOP 10 * FROM serving.vw_sales_daily;

SELECT TOP 10 * FROM serving.vw_customer_360;
```

11. CI/CD - exact steps

ADF and Synapse both support Git-based collaboration and deployment. ADF publishes ARM templates into adf_publish by default. Synapse uses a separate publish flow and supports GitHub Actions deployment for workspace artifacts.

Step 11.1 - Create a service principal for GitHub Actions

Use Azure Cloud Shell or local Azure CLI. Replace placeholders before running.

```
az login
```

```
az ad sp create-for-rbac   --name github-azure-retail-lab   --role Contributor   --scopes
/subscriptions/<subscription-id>/resourceGroups/rg-student-retail-lab   --sdk-auth
```

Copy the JSON output into a GitHub secret named AZURE_CREDENTIALS for the ADF workflow. For Synapse, you can either re-use the same principal or create a separate one and store the clientId, clientSecret, subscriptionId, and target names as individual GitHub secrets.

Step 11.2 - GitHub secrets to create

Secret name	Used by	Value
AZURE_CREDENTIALS	ADF GitHub Action	Full JSON from az ad sp create-for-rbac --sdk-auth
AZURE_SUBSCRIPTION_ID	ADF GitHub Action	Student subscription ID
TARGET_RG	Both workflows	Target resource group name
TARGET_ADF_NAME	ADF GitHub Action	adf-retail-test
CLIENTID	Synapse GitHub Action	Service principal client/application ID
CLIENTSECRET	Synapse GitHub Action	Service principal secret
TARGET_SUBSCRIPTION_ID	Synapse GitHub Action	Student subscription ID
TARGET_SYNAPSE_WORKSPACE	Synapse GitHub Action	syn-retail-test

Step 11.3 - ADF publish and deploy

- In ADF Studio, finish all changes in Git mode and publish once. This creates or updates the adf_publish branch.
- Commit the repo and push both main and adf_publish to GitHub.
- Copy cicd/adf-deploy.yml to .github/workflows/adf-deploy.yml in your repo.
- The workflow triggers on pushes to adf_publish and deploys ARMTemplateForFactory.json and ARMTemplateParametersForFactory.json to the target ADF.

Step 11.4 - Synapse publish and deploy

- In Synapse Studio, make sure publish_config.json sets workspace_publish as the publish branch.
- Publish from the collaboration branch. This writes the workspace template files into workspace_publish.
- Copy cicd/synapse-deploy.yml to .github/workflows/synapse-deploy.yml in your repo.
- Make sure workspace_publish/parameters.yaml contains the test workspace name or any environment-specific overrides you need.

12. Validation checklist

- ADF: pl_metadata_ingest completes successfully and curated folders contain Parquet files
- ADF: pl_transform_publish completes successfully and serving folders contain Parquet files
- Synapse: 01_validate_openrowset.sql returns rows from curated/orders and curated/order_items
- Synapse: serving.vw_sales_daily and serving.vw_customer_360 return rows
- CETAS: serving/sales_daily_export and serving/customer_360_export folders are created in ADLS
- CI/CD: pushing to adf_publish deploys to the test ADF
- CI/CD: pushing to workspace_publish deploys Synapse workspace artifacts to the test Synapse workspace

13. Troubleshooting shortcuts

Issue	Most likely cause	Fix
ADF linked service test fails for ADLS	ADF managed identity lacks storage role	Grant Storage Blob Data Contributor on the storage account
Synapse OPENROWSET or CETAS fails	Synapse managed identity lacks storage access or placeholder not replaced	Grant storage role and replace <storage-account> in SQL files
ADF workflow not deploying	adf_publish branch not created or not	Publish from ADF Studio and push

	pushed	adf_publish
Synapse workflow not deploying	workspace_publish branch missing or secrets incomplete	Publish from Synapse Studio and verify GitHub secrets
Data Flow consumes too much credit	Too many runs / debug sessions	Use small datasets and run Data Flow only when needed

14. Cleanup

- Delete the test ADF and test Synapse workspace after CI/CD validation.
- Delete the entire resource group if you finish the lab and do not need the environment anymore.
- If you keep the dev environment, stop creating new expensive resources and keep using serverless SQL only when needed.

15. Included files in the package

Path	Purpose
data-gen/generate_retail_data.py	Synthetic data generator
synapse/sql/00_create_db.sql	Create database and schemas
synapse/sql/01_validate_openrowset.sql	Validate curated lake reads
synapse/sql/02_views.sql	External views over curated data
synapse/sql/03_analytics_views.sql	Business views
synapse/sql/04_external_objects.sql	External data source and file format
synapse/sql/05_cetas_publish.sql	CETAS serving outputs
synapse/publish_config.json	Set Synapse publish branch
cicd/adf-deploy.yml	ADF GitHub Action
cicd/synapse-deploy.yml	Synapse GitHub Action
docs/adf_expressions.md	ADF dynamic expressions used in the lab

16. Reference notes used when preparing this guide

- Azure for Students offer and exclusions
- ADLS hierarchical namespace and storage account creation guidance
- Synapse serverless SQL overview and managed identity storage permissions
- ADF introduction, linked services/datasets, Copy activity, Mapping Data Flow, and source control guidance
- Synapse source control and CI/CD guidance
- GitHub Actions for ARM deployment